MODULE

17

Concurrency

Threads, processes and asyncio: when each one helps, and the GIL question every interview asks.

YOU WILL LEARN

Threads	`01_threads.py`
Processes	`02_processes.py`
asyncio basics	`03_asyncio_basics.py`
Demo: 30 shops at once	`04_demo_price_checker.py`
TaskGroup & queues	`05_taskgroups_and_queues.py`
Interview Q&A	`INTERVIEW.md`

WHY CONCURRENCY

Most programs spend their time waiting

THE PROBLEM

Your script checks prices in **30 online shops**. Each request takes 1 second, and 0.99 s of that is **waiting** for the network.

One after another: **30 seconds**. The CPU is idle the whole time.

THE IDEA

Start all 30 requests, then wait for them together: about 1 second. That's concurrency.

PICTURE IT CONCURRENCY

I/O-bound vs CPU-bound

🍳 A cook with 3 pots: while water boils (**waiting**), she chops vegetables. That's **concurrency**: one person, many tasks. Three cooks chopping at the same time is **parallelism**: more hands, more work done.

I/O-bound
network, disk, DB: mostly waiting

→

use
asyncio or threads

→

CPU-bound
maths, images, parsing: busy CPU

→

use
processes (all cores)

STEP BY STEP THE GIL

The GIL in one slide

	THREADS	PROCESSES	ASYNCIO
Runs Python code in parallel?	**no** (one thread at a time, the GIL)	**yes**, one per core	no (one thread)
Good for	waiting: HTTP, files, DB	heavy CPU work	**thousands** of waits
Cost per task	~MBs (an OS thread)	high (a whole interpreter)	tiny (a coroutine)
Shared memory?	yes: needs **locks**	no: data is copied (pickled)	yes, but switches only at **await**
Tool	`ThreadPoolExecutor`	`ProcessPoolExecutor`	`asyncio.gather` / `TaskGroup`

The **GIL** (Global Interpreter Lock) is released while a thread waits for I/O, so threads still help with waiting. Python 3.13+ has an experimental free-threaded build.

TRY IT THREADS

30 seconds become 1 second

```python
import time
from concurrent.futures import ThreadPoolExecutor

def fetch_price(shop):
    time.sleep(1)                       # pretend network wait
    return shop, 19.99

shops = [f"shop{i}" for i in range(30)]

start = time.perf_counter()
with ThreadPoolExecutor(max_workers=30) as pool:
    prices = list(pool.map(fetch_price, shops))
print(len(prices), "prices in", round(time.perf_counter() - start, 1), "s")
```

OUTPUT

```
30 prices in 1.0 s
```

1. **pool.map** runs the function in many threads.
2. Sequential would take 30 s.
3. Perfect for **requests**, file downloads, DB calls.

COMMON MISTAKE THREADS

Race condition: two threads, one counter

✗ WRONG

```
balance = 0
def deposit():
    global balance
    for _ in range(100_000):
        balance += 1      # read, add, write:
                          # not atomic!
# 4 threads -> often LESS than 400,000
```

Two threads read the same old value, both add 1, both write it back: one deposit is **lost**. The bug appears randomly.

✓ RIGHT

```
import threading
lock = threading.Lock()
def deposit():
    global balance
    for _ in range(100_000):
        with lock:        # one thread at a time
            balance += 1
# always 400,000
```

Protect shared data with a **Lock**, or avoid sharing: give each thread its own data and combine the results.

TRY IT PROCESSES

Use every CPU core

```python
import time
from concurrent.futures import ProcessPoolExecutor

def count_primes(limit):
    return sum(all(n % d for d in range(2, int(n ** .5) + 1))
               for n in range(2, limit))

if __name__ == "__main__":              # required for processes!
    jobs = [200_000] * 4
    start = time.perf_counter()
    [count_primes(j) for j in jobs]
    one = time.perf_counter() - start

    start = time.perf_counter()
    with ProcessPoolExecutor() as pool:
        list(pool.map(count_primes, jobs))
    many = time.perf_counter() - start
    print(f"1 process : {one:.1f} s")
    print(f"processes : {many:.1f} s")
```

OUTPUT

```
1 process : 3.4 s
processes : 1.5 s
```

1. Each process has its own interpreter and its own GIL.
2. Arguments and results are **pickled** between processes.
3. The **__main__** guard is required on Windows and macOS.

PICTURE IT ASYNCIO

async / await: one thread, many waits

☕ A **barista** takes your order, starts the machine, and while it runs takes the next order. **await** = "I'm waiting here, serve someone else meanwhile". The **event loop** is the barista.

`async def fetch()`
defines a coroutine

→

`await sleep / http`
pause and give control back

→

`event loop`
runs whoever is ready

→

`gather / TaskGroup`
wait for many at once

TRY IT ASYNCIO

Check 30 shops on one thread

```python
import asyncio
import time

async def fetch_price(shop):
    await asyncio.sleep(1)                  # non-blocking wait
    return shop, 19.99

async def main():
    shops = [f"shop{i}" for i in range(30)]
    async with asyncio.TaskGroup() as tg:  # 3.11+
        tasks = [tg.create_task(fetch_price(s)) for s in shops]
    return [t.result() for t in tasks]

start = time.perf_counter()
prices = asyncio.run(main())
print(len(prices), "prices in", round(time.perf_counter() - start, 1), "s")
```

OUTPUT

```
30 prices in 1.0 s
```

1. **asyncio.run** starts the event loop.
2. **TaskGroup**: if one task fails, the others are cancelled.
3. Real HTTP: **httpx.AsyncClient** or aiohttp, not requests.

COMMON MISTAKE ASYNCIO

Blocking the event loop

✗ WRONG

```
async def handler():
    data = requests.get(URL)      # blocks!
    time.sleep(2)                 # blocks!
    return data.json()
```

A blocking call freezes the **whole** event loop: every other request in your FastAPI app waits too.

✓ RIGHT

```
async def handler():
    async with httpx.AsyncClient() as c:
        data = await c.get(URL)
    await asyncio.sleep(2)
    return data.json()
# or: await asyncio.to_thread(blocking_func)
```

In **async def**, only use awaitable libraries. Wrap old blocking code with **asyncio.to_thread**.

IN REAL JOBS CONCURRENCY

Concurrency at work

FastAPI endpoints

```python
@app.get("/quote")
async def quote():
    return await pricing_client.get_quote()
```

Call several services in parallel

```python
user, orders = await asyncio.gather(
    users_api.get(uid), orders_api.list(uid))
```

Background jobs (Module 26)

```python
send_email.delay(order_id)    # Celery worker
```

Batch CPU work

```python
with ProcessPoolExecutor() as pool:
    pool.map(resize_image, paths)
```

INTERVIEW SAY THIS IN THE INTERVIEW

“What is the GIL? Does Python support multithreading?”

- The **Global Interpreter Lock** lets only **one thread run Python bytecode at a time** in CPython.
- It keeps reference counting simple and safe, but stops threads using several cores for **pure Python CPU work**.
- Threads **still help for I/O**: the GIL is released while waiting for the network or disk.
- For CPU work: **multiprocessing**, or C extensions like NumPy that release the GIL.
- Python 3.13+ offers an experimental **free-threaded** (no-GIL) build.

> **Follow-up they ask next:** ”Threads vs processes vs asyncio: which would you use for a web scraper? For image resizing?” (scraper: asyncio or threads; resizing: processes).

INTERVIEW SAY THIS IN THE INTERVIEW

“How does async/await work?”

- **async def** creates a **coroutine**; calling it doesn’t run it yet.
- **await** pauses the coroutine until the awaited thing is done and gives control back to the **event loop**.
- The event loop runs whichever coroutine is ready, so one thread can handle thousands of waiting connections.
- It’s **cooperative**: one blocking call freezes everyone.
- Run many with **asyncio.gather** or a **TaskGroup**.

```
async def main():
    async with asyncio.TaskGroup() as tg:
        a = tg.create_task(get_user(1))
        b = tg.create_task(get_orders(1))
    return a.result(), b.result()
```

Follow-up they ask next: ”What is a race condition and how do you prevent it?” (unsynchronised access to shared state; use locks, queues, or don’t share state).

RECAP REMEMBER THESE

Module 17 in 6 lines

1. Waiting (I/O) → **asyncio** or **threads**. Computing (CPU) → **processes**.
2. The **GIL**: one thread runs Python at a time, but it's released during I/O.
3. **ThreadPoolExecutor** / **ProcessPoolExecutor**: the easy, high-level APIs.
4. Shared data + threads = race conditions; use a **Lock** or don't share.
5. **async/await**: one thread, an event loop, thousands of tasks; **TaskGroup** to run many.
6. Never block inside **async def**; use **asyncio.to_thread**.

PRACTICE NOW

Do exercises/ex17_parallel_downloads.py: make 20 slow downloads finish in about 1 second.